

CSCE 614 COMPUTER ARCHITECTURE · MODULE 12 · LECTURE NOTES

Memory Consistency and Synchronization

Why your multithreaded code does not do what it says.

MODULE OUTCOMES

Explain sequential consistency and why no hardware implements it.

Explain store buffering and the reorderings real hardware permits.

Use memory barriers and acquire/release semantics correctly.

Explain why the compiler reorders too.

Explain why lock-free programming is difficult.

1 A result that should be impossible

```
// Initially x = y = 0

// Thread 1      // Thread 2
x = 1;           y = 1;
r1 = y;          r2 = x;
```

Enumerate the interleavings of a sequentially consistent execution. Whatever the order, both stores cannot follow both loads, so at least one load must observe a 1. The outcome $r1 = 0$ and $r2 = 0$ is impossible.

On real x86 hardware it occurs, reproducibly, within seconds. This is not a compiler bug and not a data race in the sense a race detector would flag with proper atomics — the hardware is behaving exactly as specified. The specification is simply not what sequential intuition assumes.

STORE BUFFERS ARE THE EXPLANATION

A store must acquire its cache line in Modified state, which may take hundreds of cycles under coherence (Module 11). Stalling the core on every store would be catastrophic for performance.

So stores are placed in a **store buffer** and the core continues immediately. The store drains to cache later, becoming visible to other cores at that point. Meanwhile loads proceed, reading from cache or forwarding from the local store buffer.

In the example, Thread 1's store to x may still be in its store buffer when Thread 2 loads x , and vice versa. Both read the old values. Each core sees its own stores immediately and other cores see them late — which is exactly the reordering that sequential reasoning does not permit.

2 Consistency models

A **memory consistency model** is the contract specifying which reorderings a processor may perform. It is part of the ISA.

Model	Permitted reorderings	Hardware
Sequential consistency	None. Every core observes one global order consistent with program order.	No commercially significant processor. Too slow.
TSO (total store order)	Store→load only. Stores are not reordered with other stores, and loads are not reordered with other loads.	x86-64. Relatively strong.
Weak / relaxed	Store→store, load→load, load→store, store→load — nearly anything absent explicit barriers.	ARM, POWER, RISC-V.

THE PORTABILITY TRAP THAT BITES REAL PRODUCTS

Because x86 only permits store→load reordering, a great deal of under-synchronised code *happens to work* on it.

ARM and POWER permit far more. The identical code fails — intermittently, under load, in ways that do not reproduce in a debugger.

This is why concurrency bugs have surfaced years after being written, when a product was ported to ARM. The code was always wrong; the hardware had been hiding it.

Correctness cannot be inferred from testing here. Reason from the memory model, express intent with the language's atomics, and test on weakly ordered hardware as a matter of routine.

3 Ordering, expressed

Ordering	Guarantee	Typical use
relaxed	Atomicity only. No ordering with respect to anything else.	Statistics counters read only after all threads join.
acquire (loads)	No read or write after this operation may be reordered before it.	Acquiring a lock; reading a readiness flag.
release (stores)	No read or write before this operation may be reordered after it.	Releasing a lock; setting a readiness flag.
acq_rel	Both, for a read-modify-write.	Compare-and-swap in a lock-free structure.
seq_cst	All seq_cst operations across all threads share one global order.	The default, and the right choice unless you have measured a need for something weaker.

3.1 The publish/subscribe pattern

```
// Producer
data = compute();
ready.store(true, release);

// Consumer
while (!ready.load(acquire))
    /* spin */;
use(data);
```

The release store guarantees that everything written before it in program order is visible to any thread that performs an acquire load observing that store. The acquire load guarantees that nothing after it is reordered before it. Together they establish a *happens-before* relationship across the two threads, which is precisely what makes data safe to read.

With relaxed ordering on either operation, the consumer may observe `ready == true` while `data` still holds its previous value — the stores can become visible out of order. This pairing is the foundation of nearly all lock-free code, and it is the single most useful thing to take from this module.

4 The compiler reorders too

Hardware reordering is only half the problem. Compilers reorder independent operations, keep values in registers across loop iterations, and eliminate loads they can prove are redundant — all entirely valid under single-threaded semantics.

```
// This can become an infinite loop:
while (!done) { }           // `done` is a plain bool

// The compiler proves nothing in the loop writes `done`, hoists
// the load out, and generates:
//     if (!done) for (;;) { }
```

VOLATILE IS NOT A SYNCHRONISATION PRIMITIVE

In C and C++, `volatile` tells the compiler not to optimise away or cache accesses to a variable. That is *all* it does.

It provides no atomicity: a volatile increment is still a non-atomic read-modify-write. It provides no hardware ordering: the processor may still reorder around it freely.

It is intended for memory-mapped I/O registers, where every access must actually occur, not for inter-thread communication. (Java's `volatile` is a different keyword with different, stronger semantics — a persistent source of confusion.)

Use `std::atomic` (C++) or `_Atomic` (C11). These constrain the compiler *and* emit the necessary hardware barriers.

5 What synchronisation costs

Mechanism	Uncontended	Contended
Relaxed atomic load	~1 cycle	~1 cycle
Acquire load / release store	~1 cycle on x86 (free in the common case)	Coherence traffic, hundreds of cycles
seq_cst store on x86	20–40 cycles (requires a fence)	Worse
Uncontended mutex lock/unlock	~20 cycles	—
Contended mutex	—	1,000+ cycles: futex syscall, context switch, possible scheduler delay

Note that an uncontended mutex is cheap — roughly the cost of one atomic operation. Essentially all of the cost of locking is the cost of *contention*, which means the productive response to slow locking is usually to reduce sharing rather than to abandon locks.

6 Advice

- 1 **Use a mutex** unless profiling shows it is the bottleneck. Uncontended locks are cheap and the code is obviously correct, which has enormous value.
- 2 **Reduce sharing before optimising synchronisation.** Per-thread state combined at the end removes the problem rather than making it cheaper.
- 3 **Prefer immutability and message passing.** Data that is never mutated after publication needs no synchronisation at all beyond the publication itself.
- 4 **If you go lock-free, be explicit.** Use `std::atomic` with stated memory orderings, write down the happens-before argument in a comment, and treat the code as requiring review by someone who knows this material.

- 5 **Never invent a synchronisation primitive** without a correctness argument. This is the area of systems programming where plausible reasoning is least reliable.
- 6 **Use the tools.** ThreadSanitizer finds races that testing will not. Run stress tests on ARM as well as x86.

WHY THIS IS GENUINELY HARD

Lock-free code that is subtly wrong passes essentially every test you will write for it. The failure requires a specific interleaving, on specific hardware, under specific load, and may appear once a month in production.

The discipline is therefore to reason formally rather than empirically: establish happens-before relationships explicitly, and treat a passing test suite as no evidence whatsoever of correctness.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Onur Mutlu — Memory Consistency lectures	The hardware side: store buffers, consistency models, and fences.
Paul McKenney — Is Parallel Programming Hard? (free book)	The definitive free treatment of memory ordering in practice, from the author of the Linux RCU implementation.
Jeff Preshing — blog on lock-free programming	The clearest available explanations of acquire/release semantics, with worked examples and diagrams. Start with 'Memory Ordering at Compile Time'.
Russ Cox — Hardware and Programming Language Memory Models	A three-part series covering both hardware and language models, unusually clearly.
cppreference — std::memory_order	The precise semantics, with examples. The reference to check before writing any atomic code.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement Dekker's example and run it in a tight loop on x86 until you observe `r1 == 0 && r2 == 0`. Record how long it took.
- 2 Add a `seq_cst` fence between the store and the load in each thread and confirm the outcome becomes impossible. Measure the cost of the fence.

- 3 Implement the publish/subscribe pattern with acquire/release, then with relaxed ordering on both operations. Try to observe the consumer reading stale data — on x86 you may not, which is itself the lesson.
- 4 Write a spin loop on a plain bool and confirm with the generated assembly that the compiler hoists the load. Fix it with `std::atomic` and `compare`.
- 5 Measure uncontended and contended mutex cost, and compare against equivalent atomic operations, at 1, 2, 4, and 8 threads.
- 6 If you have access to an ARM machine (including an Apple Silicon Mac or a Raspberry Pi), run the same experiments there and compare. This is the most instructive exercise in the module.
- 7 Run any concurrent code you have written under ThreadSanitizer and report what it finds.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Explain why `r1 == 0 && r2 == 0` is possible in Dekker's example, in terms of store buffers.
- 2 Why does no real hardware implement sequential consistency?
- 3 What reordering does x86 TSO permit, and what additional ones do ARM and POWER permit?
- 4 State precisely what acquire and release each guarantee, and how they combine.
- 5 Why is volatile insufficient for inter-thread communication in C++?
- 6 An uncontended mutex costs about 20 cycles. What does that suggest about when to replace locks with lock-free code?